UNIVERSITY OF CALIFORNIA
SANTA CRUZ

HARDWARE/SOFTWARE CO-DESIGN OF AN FPGA-BASED CNN ACCELERATOR

A dissertation submitted in partial satisfaction of the
requirements for the degree of

Master of Science

in

COMPUTER SCIENCE AND ENGINEERING

by

Bradley Manzo

June 2026

The work of Bradley Manzo
is approved:

DocuSigned by:
Yuanchao Xu
3D6964996B624E9...

Professor Yuanchao Xu, Chair

DocuSigned by:
2D4B17261B0A431...

Professor Dustin Richmond

Table of Contents

List of Figures

List of Equations

Abstract

Hardware/Software Co-Design of an FPGA-Based CNN Accelerator

by

Bradley Manzo

While large-scale Artificial Intelligence models increasingly dominate the computational landscape, their reliance on cloud infrastructure introduces significant latency, bandwidth, and privacy bottlenecks. Edge AI mitigates these issues by deploying application-specific models directly to local devices, yet this requires highly optimized architectures to operate within strict power and resource constraints. This thesis presents the hardware/software co-design of a Convolutional Neural Network (CNN) accelerator, utilizing an ESP32c3 microcontroller and a Lattice iCEBreaker FPGA to achieve real-time, low-power inference. The microcontroller effectively handles image preprocessing, binarization, and network communication, while the FPGA is dedicated to accelerating a heavily quantized CNN. To fit within the limited Logic Cells (LCs), DSP blocks, and BRAMs of the FPGA, the network employs quantized weights and low-precision activations trained via Unified Progressive Quantization (UPQ) and Parameterized Clipping Activation (PACT). Furthermore, this work introduces a novel architectural design space exploration methodology. By developing a highly accurate, routing-aware logic cell prediction model based on pipeline depth and flip-flop utilization, a depth-first search algorithm was utilized to automatically generate and validate optimal network topologies. The resulting system successfully demonstrates efficient, real-time edge inference, perfectly balancing the strengths of both software arbitration and custom hardware acceleration.

Chapter 1

Introduction

While Artificial General Intelligence (AGI) and Large Language Models (LLMs) currently dominate the AI landscape, their deployment remains largely restricted to datacenters. This is primarily because their generality demands massive parameter counts, requiring extensive memory and compute overhead to operate. Furthermore, as AI inputs scale from text to high-bandwidth modalities like live video, outsourcing inference by transmitting raw data to the cloud becomes unsustainable for local network infrastructure.

Conversely, application-specific models that are deployed at the edge operate with drastically fewer parameters, eliminating this network bottleneck. However their remote nature demands lower power, and therefore a highly optimized and efficient architecture. Embedded inference is highly compatible with the Internet of Things (IoT), enabling intelligent, localized data extraction by transmitting only the resulting inference, which heavily reduces network bandwidth. Accelerating inference on inherently low-compute edge devices, specifically the Lattice iCEBreaker V1.1a FPGA [4] requires full utilization of all available resources: LCs

(logic cells), DSP blocks (Digital Signal Processing), and BRAMs (Block RAMS). When paired with the ESP32c3 microcontroller [2] as shown in *Figure 1.1*, a complete, partitioned system that perfectly utilizes the distinct strengths of both software and hardware can be deployed.

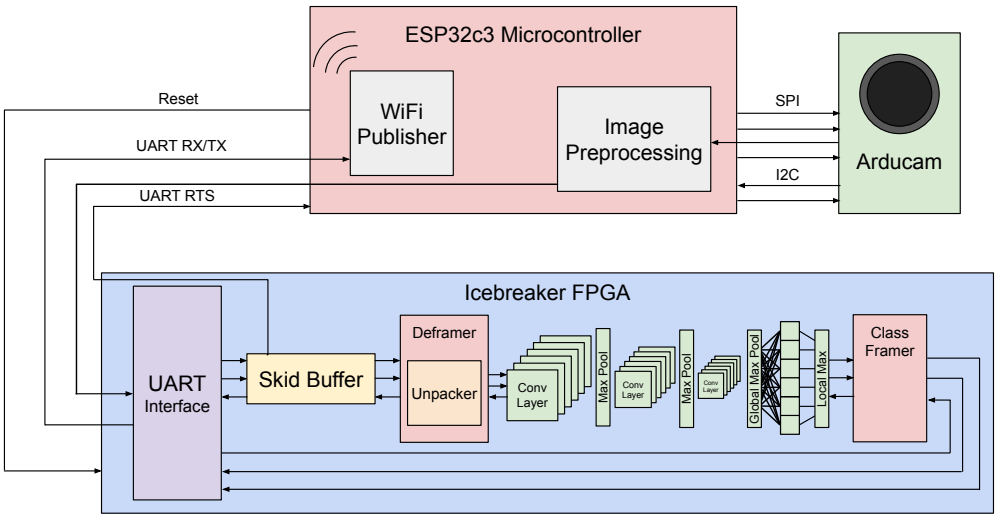

Figure 1.1: Top-Level Architecture

Chapter 2

Embedded Interface Design

The microcontroller serves two important purposes: preprocessing the Arducam image before forwarding to the FPGA, and publishing the FPGA result over Wi-Fi. Allowing the ESP32c3 to arbitrate I/O and compress the image on the front end alleviates the UART bottleneck and reduces overhead on the FPGA, ensuring that the strengths of both software and hardware are fully utilized.

Because the CNN expects binary inputs, the system leverages the YUV422 format, where every other byte directly provides a pixel's luminance. This not only reduces the memory footprint from RGB's three bytes per pixel to two, but also eliminates the costly floating point grayscale calculation. To adapt to environmental lighting changes, the system applies a dynamic threshold to binarize these values. By pressing both buttons, the threshold is set to the average lightness of the frame, however they can be pressed individually for manual adjustment.

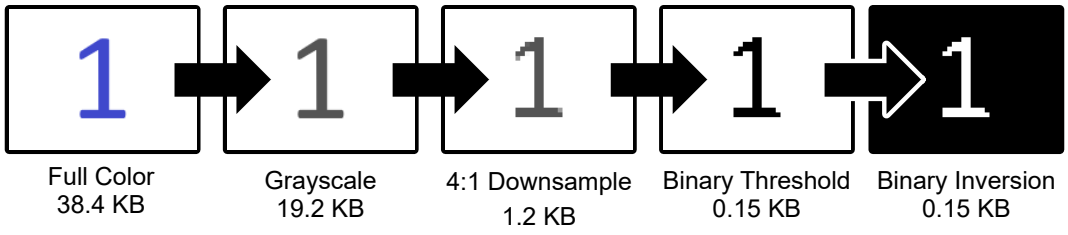

Figure 2.1: Capture Preprocessing Pipeline

To reduce SRAM consumption, the image is read off of the Arducam SPI FIFO [7] in chunks. By iteratively binarizing and packing eight pixels in each byte, the buffered image's SRAM footprint is not only minimized, but the hardware overhead is also reduced as shown in *Figure 2.1*. This compression is critical in mitigating the UART bottleneck, as the only baud rate compatible with the FPGA's 12 MHz clock frequency is 115200 baud (bits per second). The Arducam OV2640 camera captures 160x120 video at a maximum of 15 frames per second, meaning a new frame is generated every 67 milliseconds. Transmitting an unprocessed frame over UART would take 2.7 seconds reducing throughput by 40x. However, by downsampling to the 40x30 input the CNN expects and preprocessing, UART latency is reduced to 10.5 milliseconds, guaranteeing that the neural network has a continuous, real-time stream of valid data.

In order to ensure data alignment, the start of each frame is indicated by header bytes and any remaining bits are padded with zeros. The FPGA waits for these header bytes before beginning convolution and produces matching tail bytes after a valid classification, informing the ESP of its completion. System integrity is also accounted for, allowing the system to recover from interrupts or incomplete transmissions. In the absence of an FPGA response, the ESP

will periodically reset the FPGA. Upon reset, the FPGA will broadcast a wakeup byte to the ESP initiating communication. This recovery mechanism formalizes the master-slave dynamic, ensuring a robust system.

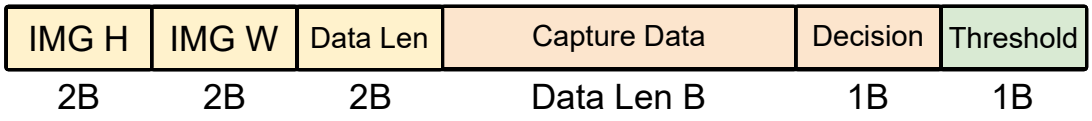

Figure 2.2: HTML Packet Structure

After the classification decision is received from the FPGA over UART, the ESP32 embeds it into a data packet alongside the binarized image, image dimensions, and the adaptive threshold as shown in *Figure 2.2*. The ESP32 acts as an HTTP web server, hosting an HTML interface and transmitting these packets over Wi-Fi to a connected client. The client's web browser unpacks the packet to visually reconstruct the binarized image for verification, while displaying the neural network's decision and the adaptive threshold used during preprocessing.

Chapter 3

Architectural Design

Computer vision is the subfield of AI focused on image processing and inferring meaning from the real world. By training neural networks on curated libraries of images, AI can effectively learn hidden spatial correlations to detect and classify subjects. While newer architectures, such as vision transformers, excel at capturing the entire image within a single global context, this comes at the cost of enormous compute and memory overhead. For constrained edge devices, inference requires an architecture that prioritizes mathematical efficiency and a minimal memory footprint. Convolutional Neural Networks (CNNs) remain the foundational standard for this domain. By applying shared weights across a local, sliding context, CNNs maximize data reuse significantly reducing memory bandwidth requirements, and making them perfectly suited for low-power hardware acceleration.

The CNN architecture consists of repeated layers of convolution followed by max pooling which selects only the highest value within each window. This combined layer detects spatial features while concentrating the output activation so deeper layers in the sequence can

form deeper correlations. After the last layer, global max propagates only the largest value in the remaining frame towards a final 1x1 convolution. Finally a comparator selects the highest value and outputs the corresponding classification, effectively translating pixels to inference.

3.1 Convolution Architecture

Convolution exploits two forms of locality: spatial and temporal. Given any input image and a 3x3 kernel, each row of the kernel window consists of three neighboring pixels. When the kernel slides from left to right across the input image it reuses columns two and three, only requiring one new column per operation. However because the kernel translates vertically as well as horizontally, the previous two rows are subject to temporal locality and reused. Streaming architectures capitalize on this behavior to produce a new convolution each cycle with only a single new input pixel by utilizing line buffers to cache the previous rows.

This massive data reuse is what sets this architecture apart; repeated memory accesses are exceedingly costly, creating a bottleneck known as the memory wall. This issue compounds when cascading layers, as each successive layer expands the receptive field, introducing an exponential data dependency. In other words, computing a deep feature without data reuse requires redundantly fetching exponentially larger windows of base pixels.

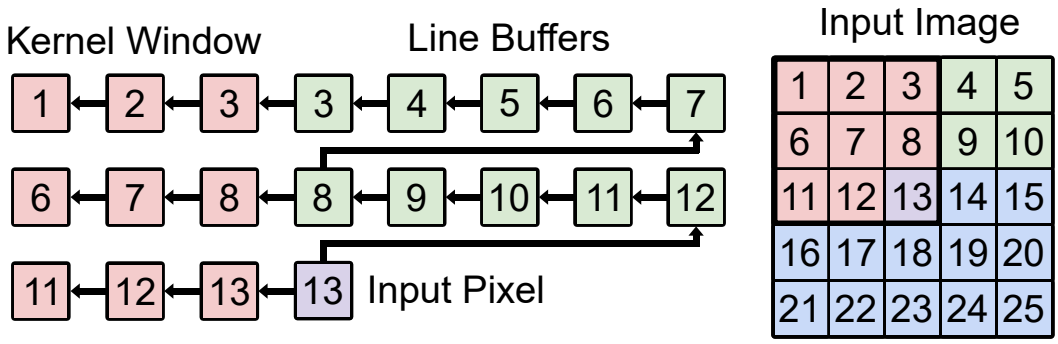

Figure 3.1: Convolution Window Data Flow

As illustrated in *Figure 3.1*, the architecture sustains a single-cycle access pattern by chaining K-1 circular buffers head to tail, where each head is tapped by column K of the kernel window. These buffers must each cache an entire row of the input image, but when images exceed 160 pixels wide, this quickly outscales a simple shift register implementation.

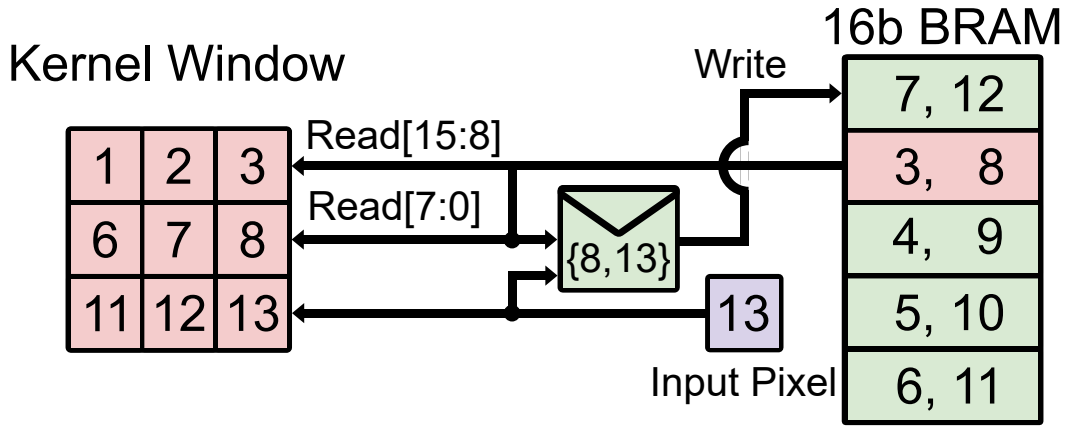

Figure 3.2: Line Buffer BRAM Implementation

Instead, line buffers are mapped directly to the Lattice iCE40's 4Kb Block RAMs (BRAMs). Because the FPGA only contains 30 BRAMs, and each natively supports a 16-bit

data width, dedicating a full BRAM for each circular buffer would rapidly exhaust these limited resources. Because inputs are often 4-8x smaller, BRAM usage can easily be halved by packing K-1 circular buffers onto a single, widened memory word. In *Figure 3.2*, a single 16-bit BRAM entry holds two 8-bit taps for a given column. During each cycle, the entry is read, the older pixel (lower byte) is combinatorially shifted to the upper byte, and the new incoming input pixel is appended to the lower byte before being written back.

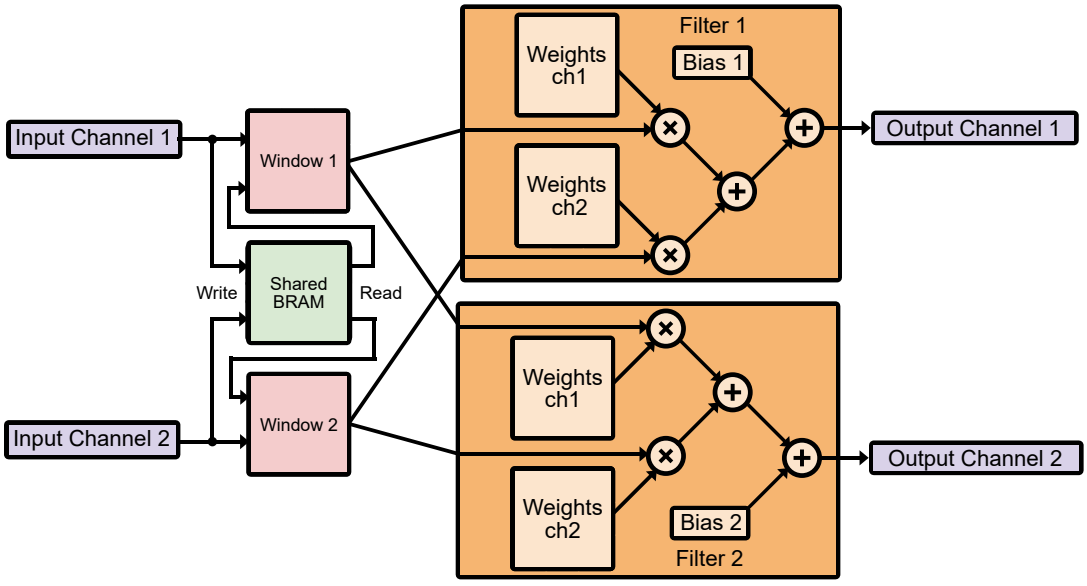

Figure 3.3: Multi-Channel Architectural Expansion

Consequently, the input image width dictates the required circular buffer depth, which in turn defines the total number of parallel RAM blocks needed. To scale this architecture, each module is heavily parameterized for any possible configuration in the CNN, allowing for any number of input and output channels, any bitwidth for weights, biases, input, and output activations. Kernel width can be configured, as well as stride and padding. *Figure 3.3* illustrates

this dimensional expansion for a configuration mapping two input channels to two output channels. To sustain the single-cycle access pattern, the RAM word is further widened to pack four row taps (two per input channel). Simultaneously, the spatial window broadcasts to two independent filter modules, each containing the two weight matrices necessary to compute the full inner product for its respective output channel. However the most critical parameter is DSPCount, which determines how many DSP multipliers to divide the workload over.

$$N_{mult} = KernelWidth^2 \cdot InputChannels \cdot OutputChannels \tag{3.1}$$

Equation 3.1: Multiplication Count

$$C_{mult} = N_{mult} \cdot InBits \cdot WeightBits \tag{3.2}$$

Equation 3.2: Multiplication Complexity

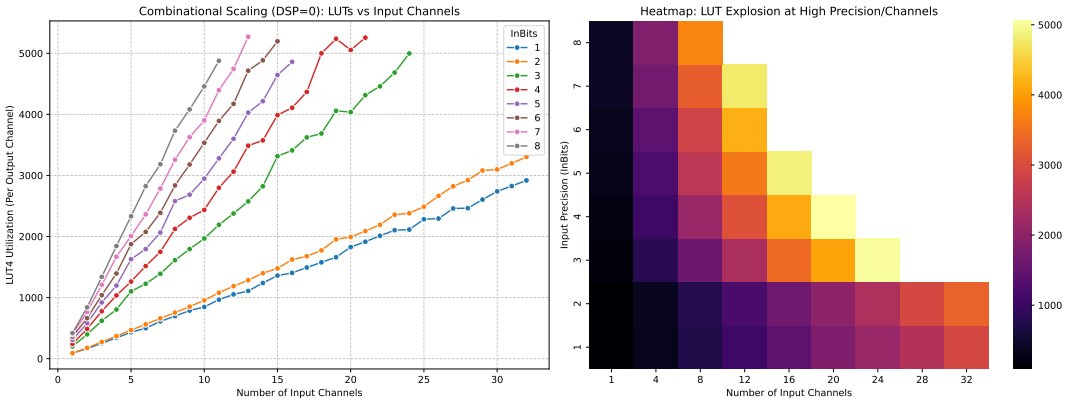

Figure 3.4: LUT4 Cost Over Multiplication Complexity

When DSPCount = 0, the layer is spatially unrolled into a fully combinational datapath and mapped entirely to LUTs (Look-Up Tables) as seen in *Figure 6*. While this configuration yields the highest possible throughput, it traditionally comes at the cost of explosive multiplication complexity as seen in *Equation 3.2*. However, this growth is linearized when weights are quantized to binary $\{-1,1\}$ or ternary $\{-1,0,1\}$ states. Because binary and ternary operations simplify to merely adding or subtracting the input multiplicand, the hardware requirement experiences a quadratic reduction in complexity.

$$N_{add} = (KernelWidth^2 \cdot InputChannels) - 1 \tag{3.3}$$

Equation 3.3: Adder Tree Count

$$C_{add} = InBits + WeightBits + log_2(N_{add} + 1) \tag{3.4}$$

Equation 3.4: Adder Tree Complexity

As a result, the dominant factor shifts from multiplication complexity, to adder tree complexity as seen in *Equation 3.4*, which scales logarithmically with the total multiplication count per output channel. This can be seen in *Figure 3.4*: when weight bitwidth is fixed to two bits, binary and ternary input bitwidths yield the lowest LUT4 cost. For this reason, designers should limit the use of fully unrolled combinational layers (DSPCount=0) to the shallower, earlier channel-sparse layers.

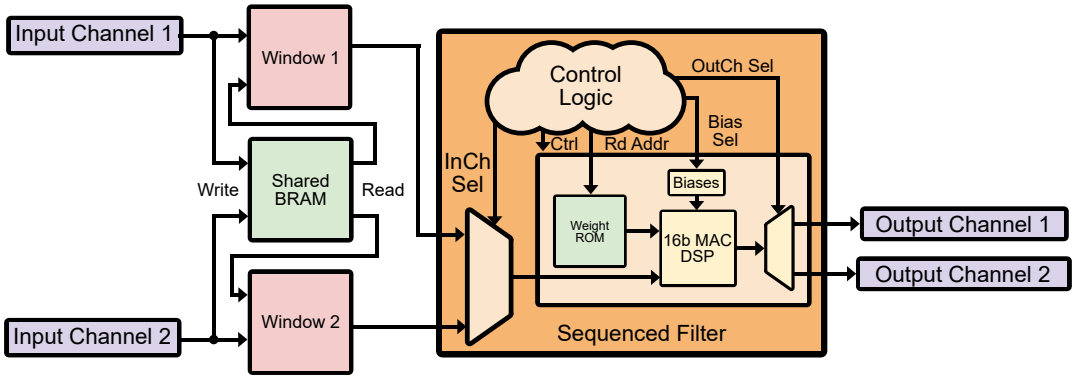

Figure 3.5: Convolution utilizing one DSP

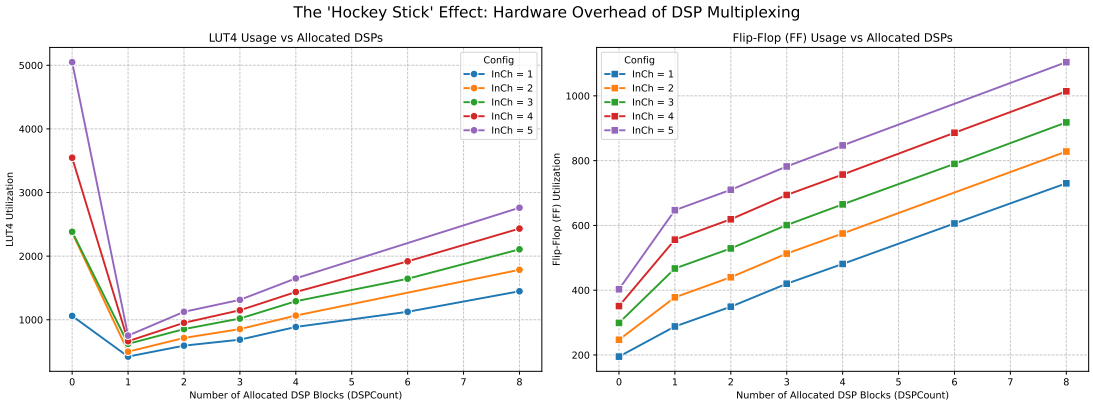

Figure 3.6: LUT4 and FF Cost over Allocated DSPs

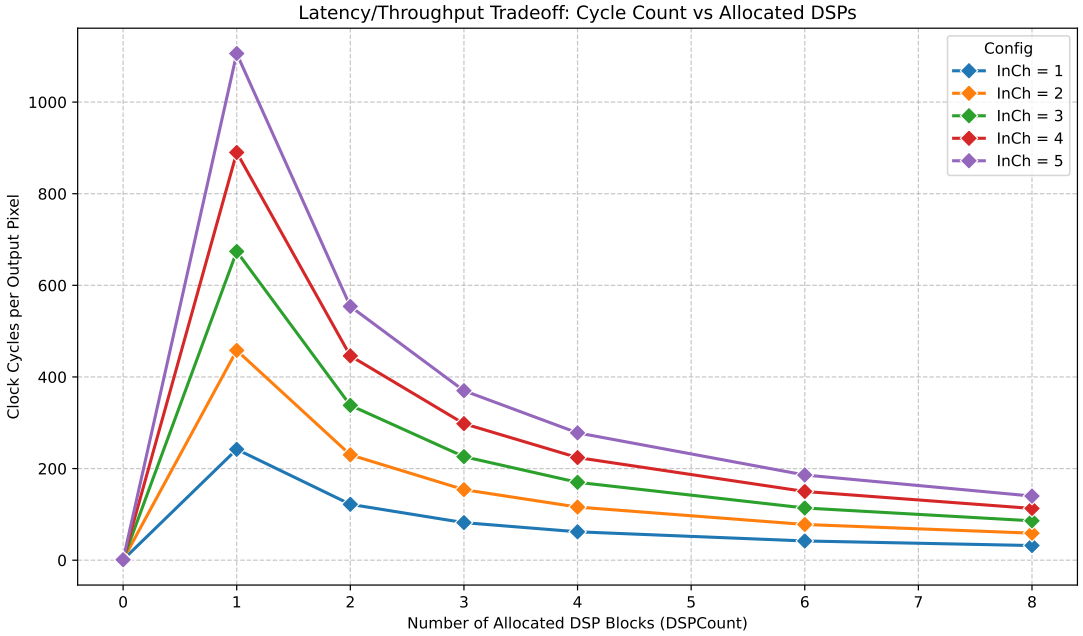

Figure 3.7: Allocated DSPs Across 24 Out Channels

For deeper, channel-dense layers, it becomes highly efficient to sequence multiplications over hard DSP blocks, resulting in a temporally folded, time-multiplexed datapath. By padding multiplicands to 16-bits and preloading the bias in the initial cycle, 32-bit Multiply-and- accumulate mode of Lattice's SB_MAC16 [5] is targeted via behavioral inference in *Figure 3.5*. While this drastically reduces combinational LUT4 overhead in *Figure 3.6*, it comes at the cost of higher FF utilization for the control logic, and higher cycle latency, as shown in *Figure 3.7*. To mitigate this latency and alleviate upstream pipeline bottlenecks, the architecture supports parallelizing the workload across up to eight distributed DSPs as seen in *Figure 3.8*. Crucially, because the datapath parallelizes across the outermost loop of the convolution iteration space, total output channels must be divided evenly among the available DSPs. Therefore,

it is strongly advised for designers to select highly divisible channel counts (e.g., 8, 12, 16) to maximize resource utilization and improve design flexibility.

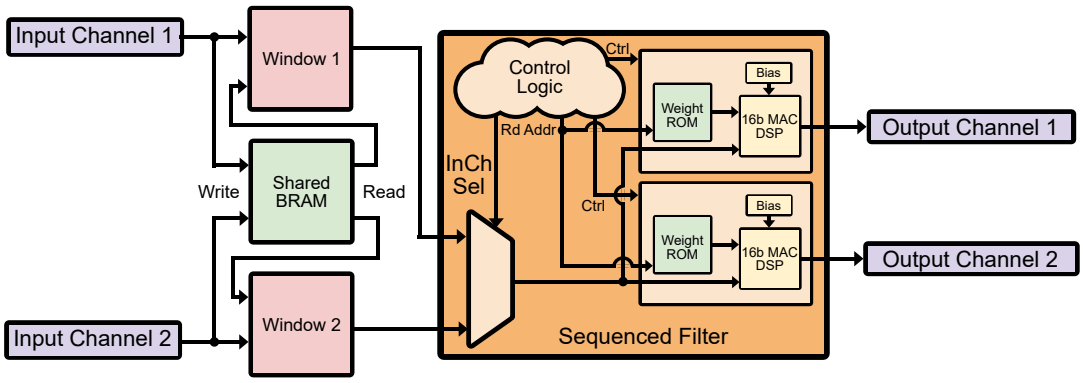

Figure 3.8: Convolution Distributed between Two DSPs

This improved channel density is not the only advantage of sequenced layers, but greater precision for virtually no cost. Because DSPs natively support 16x16 multiplication, the tradeoff for increased precision is no longer measured in LUT area, but rather in the Flip-Flop (FF) overhead required to instantiate the state-machines, sequencers, and synchronization registers that feed each physical DSP. Furthermore, because only one weight is accessed per cycle, weights no longer have to be simultaneously loaded as parameters, but can be read off of ROM when needed, effectively isolating precision from hardware cost. The remaining BRAMs can easily be synthesized as 8-bit wide ROMs for a favorable precision in these critical layers.

3.2 Pooling Architecture

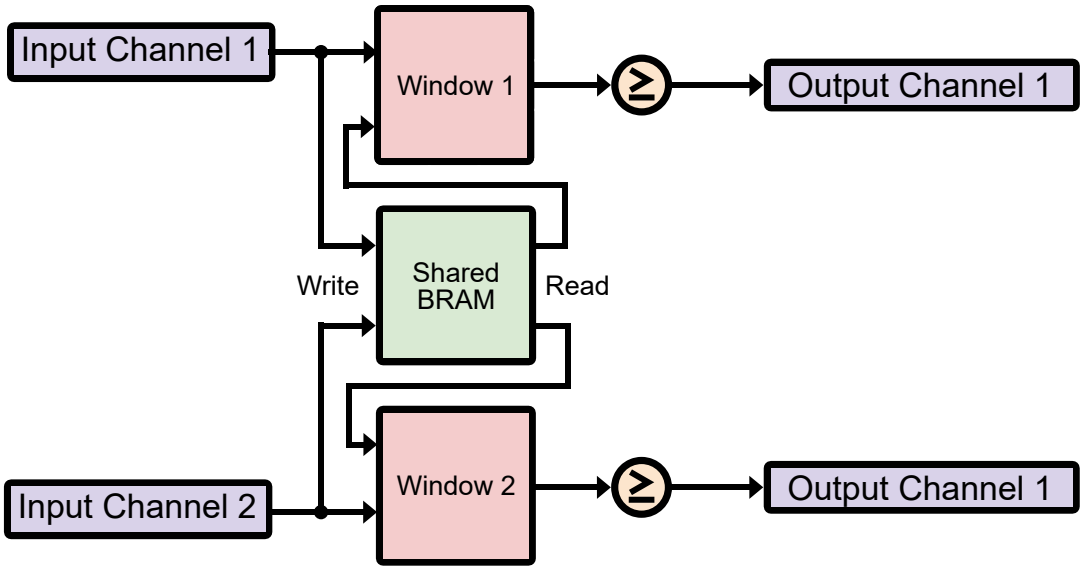

Figure 3.9: Pooling Layer Architecture

Pooling is equally important in convolutional neural networks, as it condenses learned features at each stage, reducing computational cost while increasing the spatial hierarchy of the data. Data-movement for this operation is identical to spatial convolution, however as seen in *Figure 3.9*, the arithmetic is replaced with a maximum function. Furthermore, pooling is functionally distinct because its kernel placements are mutually exclusive. For example, a 2x2 pooling kernel with a stride of two does not overlap with adjacent windows; each input pixel is evaluated exactly once, resulting in a 4:1 reduction after each pooling layer.

From a hardware perspective, finding a maximum value requires only a peak detector. An average, however, requires a division operation, which consumes significant logic area for non-powers of two. For this reason, the architecture restricts average pooling to kernel sizes

of 2x2 and 4x4, ensuring the total element count (4 and 16, respectively) is a power of two. This allows the expensive division operation to be mapped to a completely zero-cost logical right-shift in the hardware. Because maximum and average operations rely purely on logical comparators and adders, which do not benefit from the hardwired multiplication pipelines of DSP blocks, every pooling layer is completely spatially unrolled into the LUT fabric using zero DSPs. This combinational datapath incurs exceptionally small overhead with maximum throughput.

3.3 Classifier Architecture

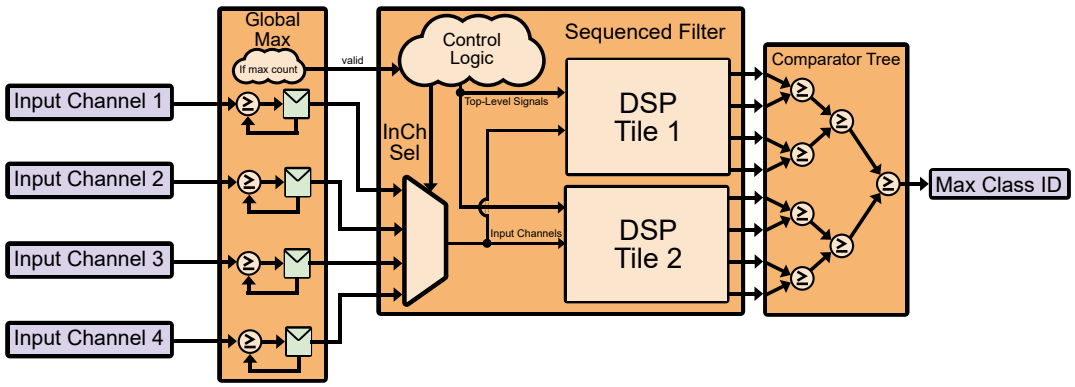

Figure 3.10: Classifier Layer Architecture

The classifier as seen in *Figure 3.10*, serves as the final stage in the CNN architecture, performing a global max pool, an intermediate linear transformation, and an argmax selection to output the maximum class ID. The global max pool extracts the maximum activation from the final spatial feature map; however, rather than instantiating the massive combinational trees used in the standard pooling layers, this stage efficiently utilizes sequential peak detectors on

each channel, reporting the highest values after WxH clock cycles. The intermediate convolution acts as a fully connected linear layer (effectively a 1x1 kernel), which is heavily advised to be allocated at least one DSP. Because the primary throughput bottleneck typically before the classifier, this provides enough slack to sequence using an idle DSP to eliminating the combinational adder trees and preserving hundreds of soft Logic Cells (LCs)

The final stage is a hardware implementation of the PyTorch argmax function (comparator) that is explicitly defined as a parallel binary reduction tree. The linear layer outputs populate the leaves of the tree, and each subsequent node combinationally propagates the maximum value and its associated ID from its two predecessors. This structured O(log2N) reduction guarantees minimal logic depth, outperforming naive implementations in both hardware complexity and timing by computing the final class ID in a single, zero-latency combinational pass.

Chapter 4

Functional Verification

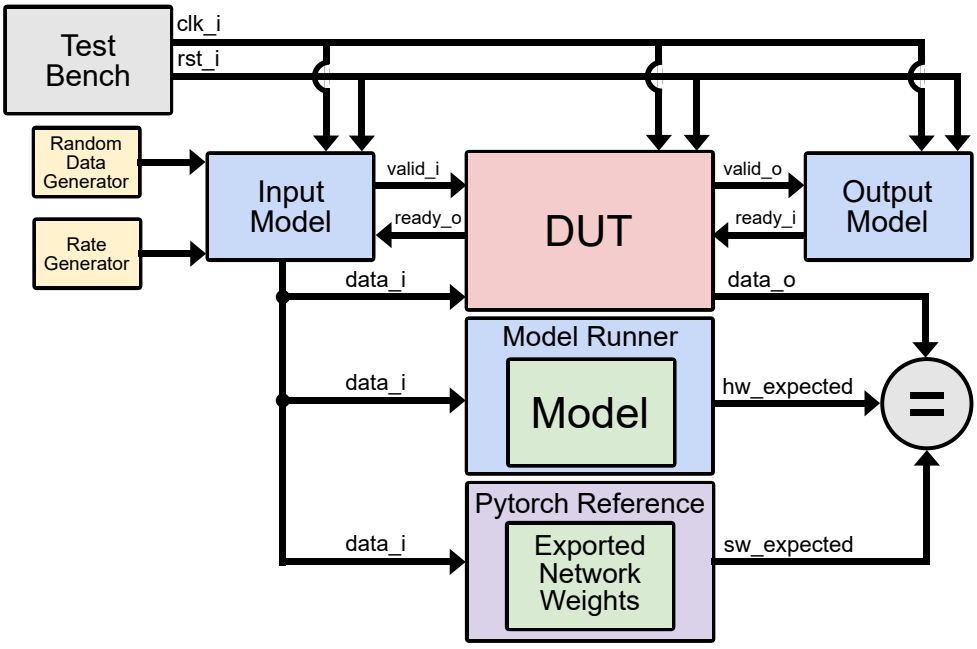

Figure 4.1: Testbench Layout

Throughout the development of each system verilog module, testbenches were simultaneously developed using the Cocotb Python library. The standardized layout as shown in *Figure 4.1* involved driving both the DUT and the golden reference from the same input model, and collecting the outputs on the output model. This structure allows verification of not only the accuracy of the data, but the timing of the hardware as well. Then when designing the individual tests, all possible edge cases were simulated along with random inputs under different combinations of throughput to verify that the module is resilient to changes in backpressure and forward pressure. For the main RTL blocks, benchmarking the outputs directly against their PyTorch equivalents guaranteed accuracy while iterating and scaling towards a complete CNN.

Once all individual modules were unit tested, they were combined into the complete neural network. Several integration tests were executed using identical weights extracted from PyTorch, verifying strict agreement between the PyTorch CNN model and the hardware testbench. The test environment was then encapsulated within models of the deframer and classifier to confirm consistent behavior. Finally, the design was integrated with Alex Forencich's UART IP [3], and the simulation was driven using Cocotb's UART protocol.

Chapter 5

Neural Network Design

Following preliminary hardware development and functional verification of the pipeline, a compatible PyTorch architecture was established and benchmarked against the MNIST database. Building upon prior testing that confirmed behavioral parity for PyTorch's conv2d and max_pool operations, the classifier was modeled using amax, linear, and argmax functions. The primary challenge in this integration involved a data precision disparity: the target edge device processes binarized inputs and integer weights, whereas PyTorch natively utilizes 32-bit floating-point representations. Consequently, a critical requirement for successful integration was quantizing these values to integers and embedding hardware-in-the-loop constraints directly into the training process to preserve model accuracy.

5.1 Weight Quantization

While weight quantization is a mandatory constraint for this edge architecture, its benefits extend equally to fully software-based models. Reducing parameter precision can dras-

tically shrink memory footprint as well as improve bandwidth, as many smaller weights can be packed into one memory entry and fetched in fewer cycles. The goal was to achieve maximum compression of weights to a ternary representation as demonstrated in Scalable MatMul-free Language Modeling [8] not just because of the aforementioned advantages, but due to the simplicity of the hardware mapping. Ternary weights are similar to binary but with an added zero term so that weights can drop out and be pruned if sufficiently small. Because $\{-1,1\}$ are only being multiplied by $\{-1,0,1\}$, the addition simplifies to a bidirectional counter. This required revision of the RTL, to not only encode binary $\{0,1\}$ to $\{-1,1\}$, but also to constrain hardware to the ternary counter logic, rather than default to the typical 2-bit range of [-2:1].

$$Q(x;s,b) = \begin{cases} s \cdot \text{clip}\left(\text{round}\left(\frac{x}{s}\right), -1, 1\right) & \text{if } b = 2 \text{ (Ternary)} \\ s \cdot \text{clip}\left(\text{round}\left(\frac{x}{s}\right), -2^{b-1}, 2^{b-1}-1\right) & \text{otherwise} \end{cases} \tag{5.1}$$

Equation 5.1: Weight Quantization

Initial experimentation revealed aggressive quantization from floating point to its final quantized precision would permanently handicap the model. The solution: Unified Progressive Quantization (UPQ) as described in Unifying block-wise PTQ and distillation-based QAT [6]. Each convolution layer is fitted with the quantization function seen in *Equation 5.1* where floating points are scaled to a lower range and then rounded to the nearest integer on forward pass.

Because the derivative of the rounding function is zero almost everywhere, standard backpropagation would effectively halt learning. To bypass this, the architecture employs

Straight-Through Estimators (STEs), which approximate the gradient of the quantization function as one. This allows the gradients to pass straight through to the underlying high-precision weights, accumulating subtle, sub-integer updates until they cross into the optimal quantized state.

Layer	FP	8b	7b	6b	5b	4b	3b	2b
1	20	5	5	5	5	5	10	20
2	30	5	5	5	5	5	10	20
3	40	5	5	5	5	5	10	20
4	50	5	5	5	5	30	30	
Classifier	50	10						

Table 5.1: Quantization Schedule

Figure 5.1: Training and Test Accuracy with Quantization Schedule

Further testing resulted in the quantization schedule described in *Table 5.1*, where the precision is gradually stepped down over several epochs. Crucially, once a layer reaches its target minimum precision (e.g., 2-bit), the precision is locked, but the network continues to train for a final set of epochs (a fine-tuning plateau). Coupled with an initially aggressive but gradually reducing learning rate, this plateau allows the weights to actively update and settle into their lowest-precision state, giving downstream layers time to recover from the upstream quantization shock. For the classifier, no further quantization beyond 8-bit was pursued, as precision here is cheaper and most critical. When inspecting the weights at each layer, earlier ternary layers become incredibly sparse and seem to outsource learning to deeper layers. Therefore, great importance should be placed in the penultimate layer, as 3-bit was the minimum quantization the model could recover from as shown in *Figure 5.1*.

Because the ternary weights and ternary activations are incompatible with each other, this introduces a critical tradeoff: reduced multiply complexity with ternary weights, or reduced datapath complexity with ternary. Although anticlimactic, this decision is made trivial thanks to the lower LUT utilization in DSP convolution layers. When optimizing logic overhead, the primary design constraint, higher precision weights are supported with no additional logical complexity. However, in earlier, thinner layers where timing is more critical, utilizing ternary activations allows for a comfortable precision of 4-bits.

$$
\begin{aligned}
BN(x) &= \gamma\frac{x-\mu}{\sigma} + \beta \\
W_{folded} &= W_{original} \cdot \frac{\gamma}{\sigma} \\
b_{folded} &= \beta - \left(\mu \cdot \frac{\gamma}{\sigma}\right)
\end{aligned}
\tag{5.2}
$$

Equation 5.2: Batch Normalization and Folding

One final weight optimization to more efficiently map to hardware is BatchNorm folding. Batch normalization is critical between neural network layers during training; by ensuring all activations have a mean (μ) of zero, and a standard deviation (σ) of one which prevents gradients from vanishing or exploding. This simplifies *Equation 5.2* to a simple scale (γ/σ) and bias β that can be folded into the already existing parameters. By multiplying the scale into each weight, and adding the offset to each bias before exporting to hardware, the network is stabilized without any costly multiplications or divisions.

5.2 Activation Quantization

While weight quantization inherently benefits any AI deployment by compressing static memory, activation quantization serves as a strict information bottleneck. By deliberately restricting the precision of intermediate feature maps, the architecture forces the network to learn robust, low-bit representations, transferring just enough meaning to sustain accuracy. On hardware, this bottleneck is what physically enables deep inference, shrinking the required datapath width and minimizing the BRAM required for temporal buffering.

After training many different architectures, none were able to surpass 50% accuracy with a full ternary datapath. While the bounded nature of ternary activations is highly beneficial for hardware, strictly limiting datapath width and making worst-case accumulator overflows highly unlikely, this flat gated function causes gradients to vanish before they can propagate back to the network's early layers. ReLU, on the other hand, is positively unbounded. By applying a constant derivative of one to positive inputs, it acts as a lossless highway, ensuring that even the earliest layers receive strong error signals to learn from.

$$y = \text{PACT}(x, \alpha) = \begin{cases} 0 & x \leq 0 \\ x & 0 < x \leq \alpha \\ \alpha & x > \alpha \end{cases} \tag{5.3}$$

Equation 5.3: Parameterized Clipping Activation (Quantization)

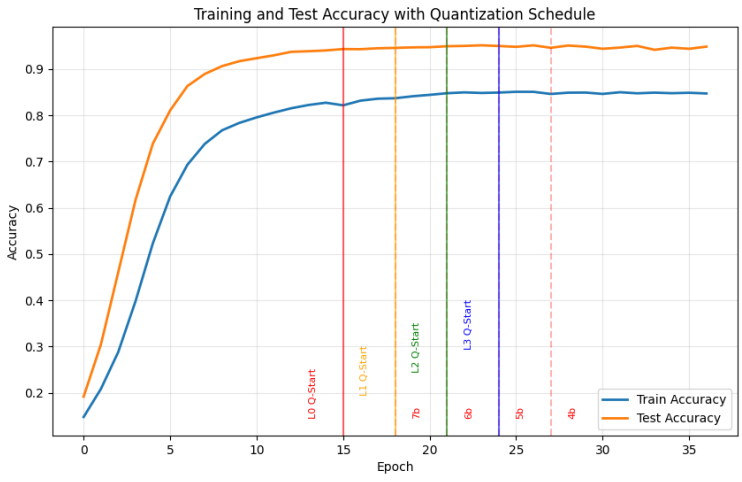

Figure 5.2: Training MNIST with Quantized Activations and Weights

It's clear that an output activation with a bit-width greater than 2-bits should function similarly to a clipped ReLU, however it's not immediately obvious which bits to select off of the sum, which in the worst case accumulate far beyond the target output width. If sufficiently smaller than the accumulation width, selecting off the LSB results in noise, while selecting off of the MSB results in zeros. Choi, J et al. bridges this gap in PACT: Parameterized Clipping Activation for Quantized Neural Networks [1] where a window with a lower bound of zero, and an upper bound α. As shown in *Equation 5.3*, everything below is zeroed, and everything above saturates to α. By forcing the network to learn an α that is a power of two, the quantization scale factor can be mathematically mapped to a hardware right-shift (dividing by 2^S). This allows the network to autonomously learn and select the most meaningful 'sweet spot' range of bits directly from the accumulator, resulting in the accuracy shown in *Figure 5.2*.

In hardware, this translates to a single static shift parameter per layer to apply the learned scale, naturally enforcing the clipping boundary α via the physical limitations of the output bit-width. While learning a custom scale per channel may yield a slight increase in accuracy, because DSPs operate over a range of out channels, dynamically shifting the output would incur massive control logic overhead. Post training, the network then observes the learned weights and produces a data-dependent worst case accumulation bound. The hardware which sets the register bitwidth to the absolute worst case is trimmed to this new width to reduce FF and LUT overhead.

Chapter 6

Hardware-Software Co-Design

At every stage of training, it is critical that data is formatted identically to its hardware equivalent. From dataset preprocessing to FPGA synthesis great care must be taken to prevent any divergence in representation. The co-design environment, depicted in *Figure 6.1*, achieves this by establishing a central consistent neural network configuration as a single source of truth.

6.1 Data Flow and Centralization

After training concludes, quantized weights are exported to a csv, and then translated to hardware according to each layer's architecture. Combinational layers require all weights to be loaded simultaneously, so they are saved as a packed array within a verilog header (.vh) file along with each layer's biases and shifts. Sequential DSP layers only need one weight per DSP per cycle, so these weights are exported to hex files to initialize each ROM. This ensures that the Python training loop, the hardware emulator, and the Verilog parameters all inherit a mathematically consistent neural architecture.

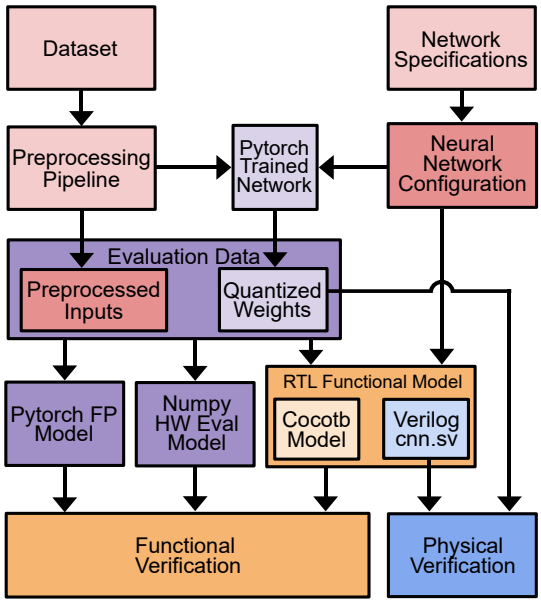

Figure 6.1: Verification Data Flow and Model Equivalency

After PyTorch training concludes, the network outputs the golden Quantized Weights which are then exported and translated according to each layer's hardware architecture. Combinational layers require all weights to be loaded simultaneously, so they are saved as a packed array within a Verilog header (.vh) file, alongside each layer's biases and learned right-shifts. Conversely, sequential DSP layers only require one weight per DSP per cycle, so these weights are exported to .hex files to physically initialize the FPGA ROMs.

To guarantee model equivalency, these quantized weights along with preprocessed inputs are simultaneously sourced by three distinct verification engines. First, the RTL Functional Model utilizes Cocotb integration tests wrapped around the instantiated Verilog to confirm that the hardware behaves as expected and produces the correct cycle-accurate decisions at the final output.

Next, to provide an additional layer of verification at scale, the evaluation data is read back into a custom, bit-accurate software emulator (hw_eval). Rather than relying on the floating-point PyTorch graph, this emulator executes pure integer matrix mathematics in NumPy, complete with explicit hardware bit-shifts and simulated accumulator overflows. While the Cocotb integration tests are essential for verifying cycle-accurate RTL behavior, the simulation is inherently slow. The hw_eval emulator bridges this gap by rapidly evaluating the entire dataset through the exact mathematical constraints of the datapath, efficiently predicting the model's true accuracy on the FPGA prior to physical synthesis.

By evaluating the dataset identically across all environments, any divergence between the theoretical PyTorch accuracy and the final hardware implementation can be quickly identified. Though hw_eval produces a hardware-accurate prediction, it does not mathematically guarantee the exact same accuracy as the software network. While in some cases the hardware accuracy is slightly greater by chance, profiling across 822 architectures reveals an average accuracy drop of -3.71% with a standard deviation of 10.76%. This variance is primarily driven by outlier networks that suffer catastrophic quantization collapse, likely due to extreme accumulator clipping or integer rounding errors that PyTorch's Straight-Through Estimators (STEs) couldn't perfectly model during training. In this case it is recommended to simply retrain the network, yielding a high likelihood of favorable quantization.

6.2 Design and Environment Flow

When prototyping the initial CNN, the lengthy design process was generalized for repeatability. Network selection, training, quantization, synthesis, programming, and deploying each introduce bottlenecks which must be evaluated properly as seen in *Figure 6.2*.

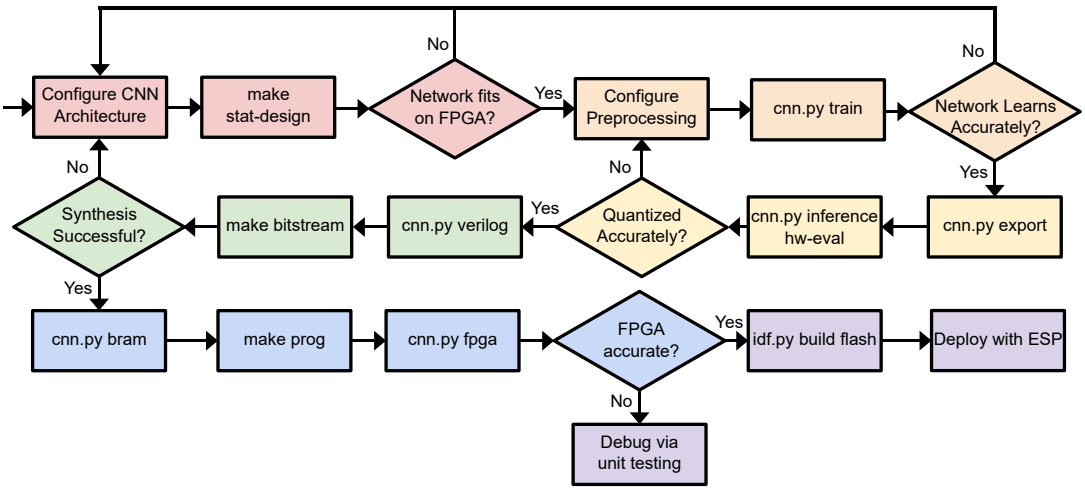

Figure 6.2: Hardware-Software Co-Design Flow

After specifying the desired network architecture (NNConfig), designers can execute a pre-flight synthesis check (make stat-design) which uses Yosys to rapidly estimate the hardware footprint. A primary challenge when targeting a resource-constrained FPGA (such as the 5280-LC iCE40) is the immense time investment required for end-to-end architectural design, PyTorch training, quantization, and final Verilog synthesis. Often, after hours of training, the final model would fail to physically fit on the device, and worse, fail to report any meaningful bottleneck metrics. This friction motivated the automated design space exploration discussed in the next section.

To bypass this bottleneck, stat-design performs a preliminary estimation by synthesizing each layer in isolation, allowing it to report the aggregate Logic Cell (LC) utilization even if the total design massively exceeds the physical hardware cap. Because this rapid approach operates before training has even begun, it lacks real learned weights, which are impossible to predict. To compensate, the script automatically injects a pseudo-random, non-zero bit pattern into the weights; this prevents the logic synthesizer from aggressively optimizing away the LUT-based multipliers due to constant-propagation. While this generates a highly accurate footprint for the isolated macros, it intrinsically ignores top-level routing and control overhead. Consequently, the resulting LC cost serves as a critical ballpark estimate, allowing designers to iteratively scale their architecture before ever committing to the lengthy PyTorch training pipeline. The greatest parameters designers can tweak here are the allocation of DSPs between layers, as well as the activation bit widths. Assigning a single DSP to deeper layers results in the least complex design possible, while activation bit widths in earlier layers can be reduced to two or three without any meaningful drop in accuracy.

After the design is settled, designers specify preprocessing and the hyperparameters involved with training the network. Task-specific macros such as input dimensions, input bit width, and number of classes are specified within the design itself. If the design does not achieve the desired accuracy, designers should prioritize increasing channel count, designing with lower initial precision and maximizing bandwidth (activation bitwidth x out channels) in the penultimate layer and classifier.

Once the desired accuracy is achieved the learned weights can be exported using cnn.py export and quantized accuracy can be tested using cnn.py inference hw-eval. This

aforementioned model allows designers to simulate how the model mathematically operates in hardware with perfect bit-accuracy. If there's a sharp drop, it's recommended to retrain and requantize.

When the quantized accuracy is within 4% of the floating point accuracy, it is recommended that designers proceed to synthesis. Cnn.py verilog references the same design file to arrange and wire the equivalent macros in cnn.sv, as well as import the required weights and parameters learned during training. This program also updates the input dimensions in the top-level and properly updates the names and amount of hex files to be imported at each layer. Running "make bitstream ESP=0" targets the iCEBreaker V1.1a for use with USB serial IO for physical verification. If synthesis does not succeed, thanks to stat-design the actual LC should still be low enough to provide a meaningful error message.

Once successfully synthesized, designers can confirm that the hex files were correctly interpreted as ROMs by using cnn.py bram which parses the synthesis logs for matching values. After programming the FPGA, the accuracy can be physically verified using cnn.py fpga, driving the FPGA using UART over the USB Serial IO rather than from the ESP. This allows us to inject preprocessed images from the dataset and directly compare the result to hw-eval.

Finally when hardware accuracy has been verified, designers can finally run "make bitstream ESP=1" to enable the top-level to read IO over the GPIO pins from the ESP32c3. All that's before deploying is updating the image capture dimensions in the firmware, and updating the web server to properly interpret the received ID.

Chapter 7

Design Space Exploration

While this co-design flow enables rapid and informed iteration via manual trial-and-error, the strict hardware constraints of the FPGA lend themselves to an exhaustive design space exploration. Not only does this save design time, but it also provides us the globally optimal network, balancing throughput, LC cost, and network accuracy.

In order to determine all possible networks that fit within the 5280 LC, 30 BRAM, and 8 DSP design constraints, a depth-first search is performed, iterating over all possible network lengths within budget. However, running make stat-design at each branch of the search would quickly scale out of control. For this reason, replacing this dynamic synthesis with a lightweight analytical model is ideal for on-the-fly LC estimations.

7.1 Convolution Prediction

For every combination of parameters in the convolution, pooling, and classifier layers, a realistic range was swept via isolated Yosys runs, terminating when exceeding 5280 LC.

To create a fast analytical model, the resulting resource utilizations (LUTs and FFs) from the synthesis logs were fitted via linear regression. Experimentation revealed that the bit-width precision costs (input and weight bit-widths) were mathematically independent from the spatial multiplication count. By extracting unique coefficients for each precision corner, the resulting regressions yielded an R2 ¿ 99%.

$$LUT4(InBits, WeightBits) = A \cdot (OutCh \cdot InCh) + B \cdot OutCh + C \cdot InCh + D \tag{7.1}$$

Equation 7.1: Convolution LUT4 Prediction

$$FF(InBits, WeightBits) = A \cdot (OutCh \cdot log_2(InCh)) + B \cdot OutCh + C \cdot InCh + D \tag{7.2}$$

Equation 7.2: Convolution FF Prediction

Equations 7.1 and 7.2 demonstrate that in the combinational unrolled layers, there are constant, predictable overheads for the spatial multiplication count (A), routing the output channels (B), routing the input channels (C), and a fixed baseline to implement control logic (D). Flip-flops can be predicted using a similar equation; however, because the input channels are summed using a pipelined binary adder tree, the number of staging registers grows logarithmically. The same functions hold for the sequential DSP layers, however since weight bits are fixed, DSPCount replaces it as the independent predicting variable. Because the LUT-based spatial multipliers are replaced with a single DSP, the A coefficient plummets as shown in Figure 9. The remaining LUTs are utilized mainly for routing overhead, while FFs increase due

to the added data staging and control logic required to drive the DSP in 16x16 multiplication mode.

7.2 Classifier Prediction

Since the classifier layer contains its own 1x1 convolution, the predicting *equations 7.3 and 7.4* resemble those of the convolution module, modeling multiplication cost using input channels, class count, and weight bitwidth. However, the additional global maximum pooling and its underlying comparator tree must be modeled as well. While the spatial storage required for the global pooling scales linearly with the input channels, the comparator tree performs a binary reduction over the class outputs, meaning its hardware footprint scales logarithmically with the class count.

$$LUT4(TermBits, DSPCount) = A \cdot (InCh \cdot ClassCount \cdot WeightBits) + B \cdot InCh + C \cdot log_2(ClassCount) + D \tag{7.3}$$

Equation 7.3: Classifier LUT4 Prediction

$$FF(TermBits, DSPCount) = A \cdot InCh + B \cdot ClassCount + C \cdot DSPCount + D \tag{7.4}$$

Equation 7.4: Classifier FF Prediction

7.3 Pooling and Overhead Prediction

Because pooling layers lack any weights, and due to channels and precision remaining fixed between input and output, hardware prediction can be simplified to a standard look-up table for all realistic combinations of input bits, input channels, and pooling mode.

Overhead measurements included the UART IP, skid buffer, and class framer, while the deframer required sweeping over the four possible input sizes on the 8 bit UART bus.

7.4 Depth-First Search Network Generation

FPGAs are composed primarily of Logic Cells (LCs), which serve as the fundamental building blocks of the architecture. On the iCE40, each LC contains a 4-input Look-Up Table (LUT4), a dedicated Flip-Flop (FF), and optional carry logic. During synthesis, high-level hardware descriptions are mapped down into these 5,280 available cells. Crucially, the LUT and FF counts reported by the synthesizer are not mutually exclusive. The architecture is designed to pack efficiently: if a combinational logic path (LUT) directly feeds into a staging register (FF), the synthesizer will merge both operations into a single shared Logic Cell. Consequently, the total Logic Cell utilization is rarely the strict sum of the two resources, but is instead tightly bounded by the maximum of the two ($LC \approx max(LUT4, FF)$).

Using this predictive LC estimate, a depth-first search of the design space was performed. The algorithm initialized from a set of realistic architectural base cases as seen in *Algorithm 7.1*, and iteratively extended the network depth with *Algorithm 7.2*. Once the predicted hardware footprint reached the physical LC capacity (minus a safety margin for top-level

routing overhead), the search terminated that branch by appending the final classifier layer.

Algorithm 7.1 Generate Base Case Networks

$S \leftarrow \emptyset$ ▷ Initialize empty set of networks
for all $oc \in \{2,4,6,8,9,10,12\}$ **do** ▷ Evenly divisible for DSPs
  for all $ob \in \{2,3,4,5,6\}$ **do**
    $ib \leftarrow 1, ic \leftarrow 1$ ▷ Binary, single channel image format
    $wb \leftarrow 4$ ▷ Optimal precision vs. lc cost
    $dsp \leftarrow 0$ ▷ Single-cycle Layer 0 for throughput
    $lc_{conv} \leftarrow \text{predict\_conv}(ic, oc, ib, wb, dsp)$
    $lc_{pool} \leftarrow \text{predict\_pool}(ib = ob, ic = oc, mode = max)$
    $lc_{total} \leftarrow lc_{conv} + lc_{pool}$
    if $lc_{total} \leq \text{LC\_CAP}$ **then**
      $cfg \leftarrow \text{build\_cfg}(oc, wb, ob, dsp)$
      $S \leftarrow S \cup \{(lc_{total}, cfg)\}$ ▷ Store valid network in set
    end if
  end for
end for
return $\text{sort}(S)$

Algorithm 7.2 Generate Networks via Depth-First Search

$S \leftarrow \emptyset$ ▷ Initialize empty set of complete networks

function EXTEND(L, lc_{used}, dsp_rem)

  $(oc_{prev}, ob_{prev}) \leftarrow$ last layer of L

  for all $oc \in \{c \in \{2,4,6,8,9,10,12\} \mid c > oc_{prev}\}$ **do** ▷ Grow channels

    for all $ob \in \{w \in \{2,3,4,5,6\} \mid w \geq ob_{prev}\}$ **do** ▷ Grow bit-width

      $wb \leftarrow 8$ ▷ ROM Width

      $dsp \leftarrow$ allocate_dsp(...) ▷ Assign DSP if $dsp\_rem \geq 1$

      $lc_{conv} \leftarrow$ predict_conv($ic = oc_{prev}, oc, ib = ob_{prev}, wb, dsp$)

      $lc_{pool} \leftarrow$ predict_pool($ib = ob, ic = oc, mode = max$)

      $lc_{total} \leftarrow lc_{used} + lc_{conv} + lc_{pool}$

      if $lc_{total} \leq$ LC_CAP **then**

        $L_{new} \leftarrow L \cup \{(oc, wb, ob, dsp)\}$

        $S \leftarrow S \cup \{(lc_{total}, \text{build\_cfg}(L_{new}))\}$ ▷ Store candidate

        if spatial dimensions ≥ 2 **then** ▷ Minimum pool size

          EXTEND($L_{new}, lc_{total}, dsp\_rem - dsp$) ▷ Recurse deeper

        end if

      end if

    end for

  end for

end function

▷ — Main Execution —

for all base layer $L_0 \in$ Generate Base Case Networks() **do**

  EXTEND($\{L_0\}$, lc_{L_0}, DSP_CAP)

end for

$S \leftarrow$ deduplicate_by_architecture(S) ▷ Keep lowest LC DSPs

$S \leftarrow \{net \in S \mid |net.layers| \geq 2 \text{ and } lc \leq (\text{LC\_CAP} \cdot 0.9)\}$

return sort(S)

To aggressively prune the search space and filter out inherently suboptimal architectures, several structural heuristics were enforced during traversal. For each successive layer, the activation bit-width and channel count were constrained to be monotonically increasing (preventing information bottlenecks), and channel dimensions were restricted to highly divisible integers (2, 4, 6, 8, 9, 10, 12) to allow for better DSP allocation. Furthermore, the first layer was strictly enforced as a purely combinational block with 4-bit weights, whereas all succeeding layers were mapped to sequential architectures utilizing 8-bit weights and a single allocated DSP macro. Maintaining a combinational layer at the network's head supports high initial throughput where spatial resolution is highest, while limiting deeper sequential layers to a single DSP ensures maximum LUT efficiency, preserving the flexibility for designers to optimize throughput later by allocating additional DSPs if the budget allows. These topological constraints align directly with the empirical findings of the previous section, drastically reducing search time while consistently yielding high-quality, physically realizable networks. This search resulted in roughly a thousand valid networks that were each synthesized, reporting the actual LC cost.

7.5 Logic Cell Prediction

While this theoretical packing holds true for small, localized designs, it was discovered that larger networks, which theoretically fit within hardware caps, were often unable to be physically routed by Nextpnr. As routing congestion grows between disparate modules, the toolset is often forced to duplicate logic to meet timing constraints, breaking the ideal 1:1 packing assumption. Because an accurate routing-aware assessment of LC is critical for a proper

design space exploration, end-to-end syntheses were conducted over a diverse dataset of complete neural networks to extract their actual, post-route LC costs. By evaluating the aggregated LUT and FF counts across the network, regression analysis was performed to determine which resource served as the dominant predictor of final hardware utilization.

The results were striking: once the heavy arithmetic logic was absorbed by the DSP macros, the predicted LUT count lost nearly all of its predictive power. Instead, the total Flip-Flop count, which serves as a direct proxy for pipeline depth and interconnect width, proved to be an exceptionally strong indicator of routing congestion. A single-variable regression on the Flip-Flop count yielded an R2 of 0.97 across 191 fully-routed networks. The maximum outlier observed being -0.09 is reflected in allocating 9% of the logic cap towards unused headroom. Ultimately, the final, routing-aware Logic Cell cost can be accurately modeled in *Equation 7.5*.

$$ActualLC = 2.74 \cdot predictedFF + 35.08 \tag{7.5}$$

Equation 7.5: Logic Cell Prediction

With this robust, routing-aware heuristic, the depth-first search can confidently prune architectural configurations that would theoretically fit, but practically fail during the physical Nextpnr routing phase.

7.6 Network Accuracy Prediction

The final phase in the design space exploration was predicting pre-training accuracy from only the neural network architecture. The previously described improvements to the DFS resulted in 964 valid networks, which were iteratively trained over a standardized schedule. Many predicting variables were experimented with including total bandwidth, overall network growth, and throughput. However the three greatest predictors were the total LC utilization, ternary datapath percentage, and network depth yielding a pretraining predictive capability of 0.53 R2 as described by *Equation 7.6*.

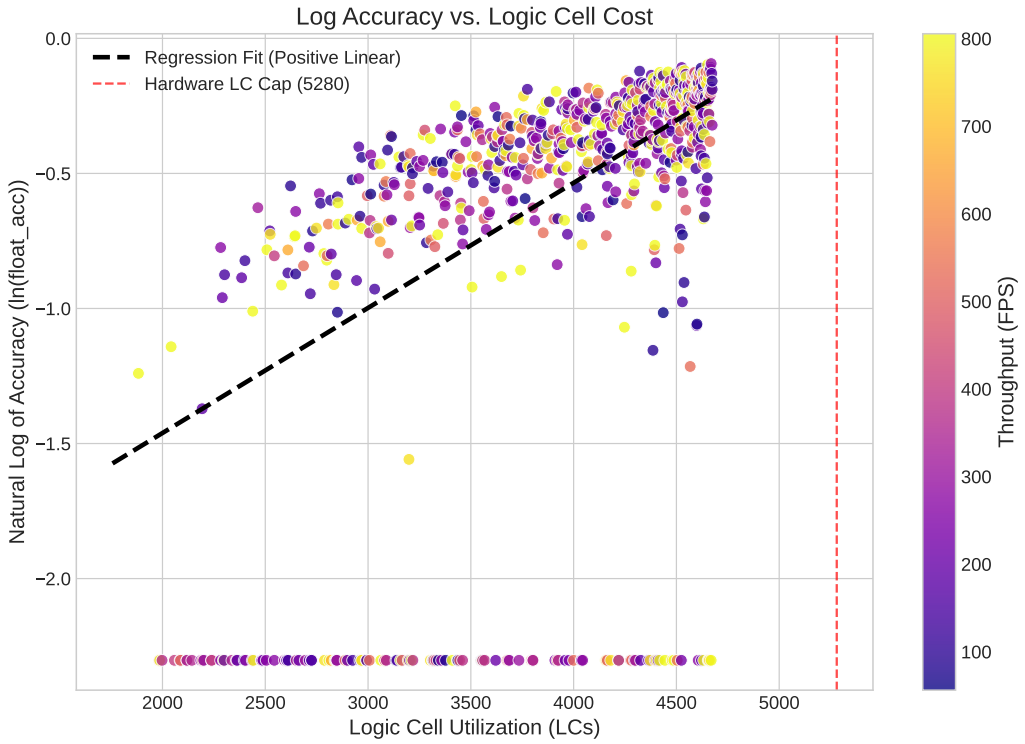

Figure 7.1: Network Accuracy vs LC Cost

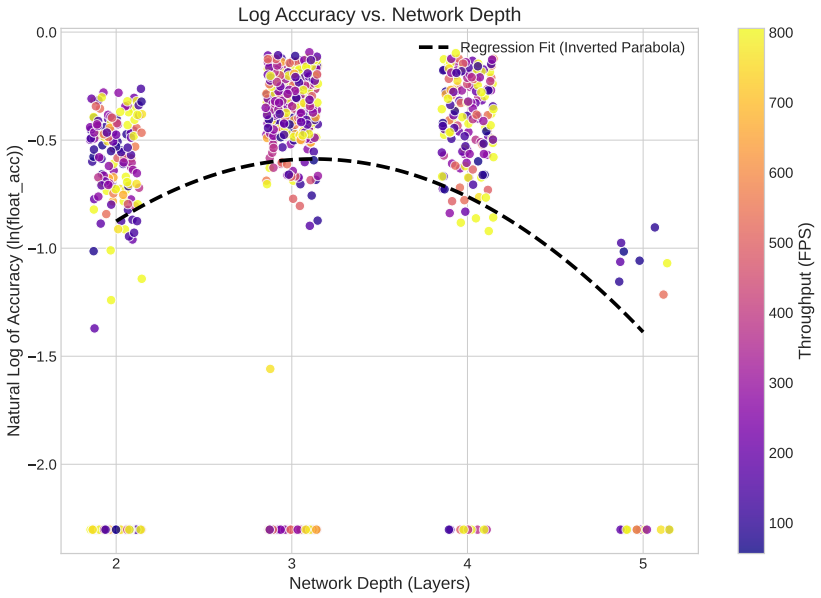

Figure 7.2: Network Accuracy vs Network Depth

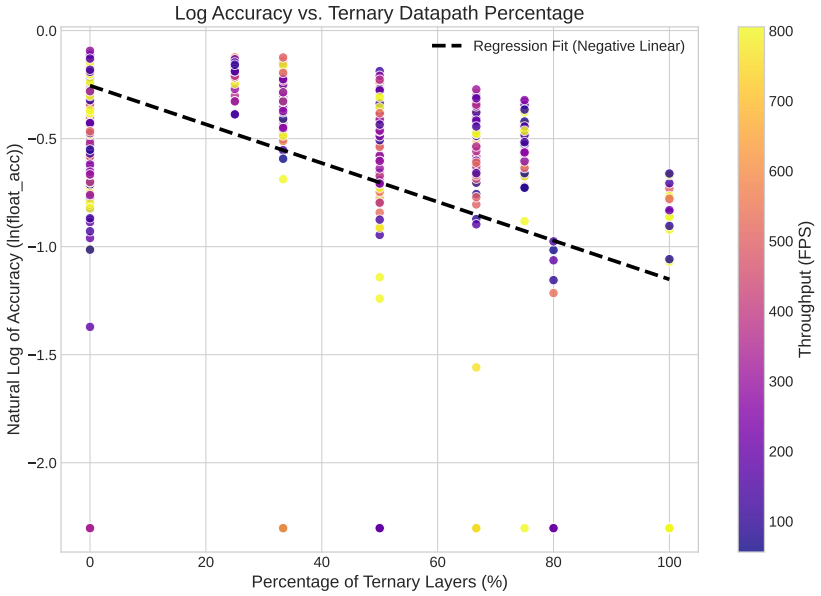

Figure 7.3: Network Accuracy vs Ternary Datapath Percentage

In *Figure 7.1*, the positive linear relationship between trained floating point accuracy and LC is clear. Unsurprisingly, networks that fully utilize all available resources perform better, however accuracy peaking at a network depth of three in *Figure 7.2*, indicates that allocating the logic cells towards fewer wider layers outperforms longer thinner networks. The reason why is displayed in *Figure 7.3*, as longer networks tend to have critically imprecise activations, relying too heavily on ternary activations, which while useful in early layers, lead to vanishing gradients when utilized in deeper layers.

$$ln(acc) = \frac{LC}{2500} - \frac{Ternary\%}{117} + 1.422 \cdot depth - 0.226 \cdot depth^2 - 4.24 \tag{7.6}$$

Equation 7.6: Float Accuracy Prediction

From a designer's perspective, throughput is just as critical for deploying a live model, as introducing downstream bottlenecks directly affects the entire system. While not the primary metric being measured by this statistical model, the structural diversity of generated networks in terms of depths, channel growth rates, and bit-widths yields a wide range of throughputs the designer may select from. Because throughput (frames per second) is largely independent of accuracy but entirely dependent on architecture, this variety enables designers to optimize for accuracy and from the top ranking candidates, select the network with the highest throughput.

Chapter 8

System Deployment and Future Work

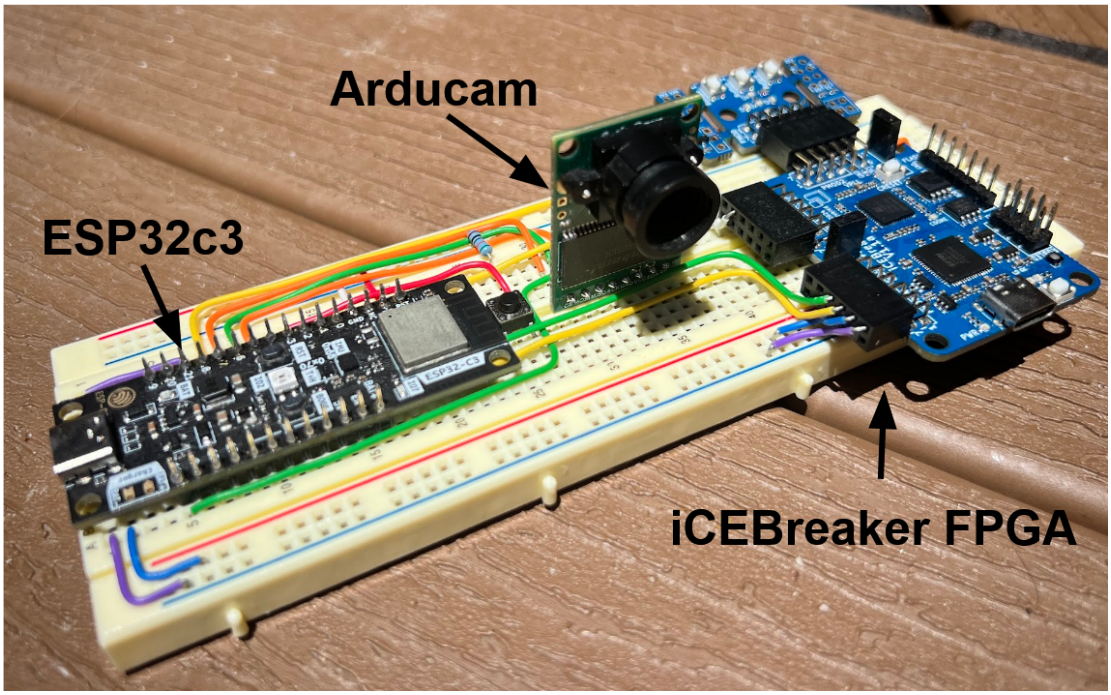

Figure 8.1: Deployed and Integrated System

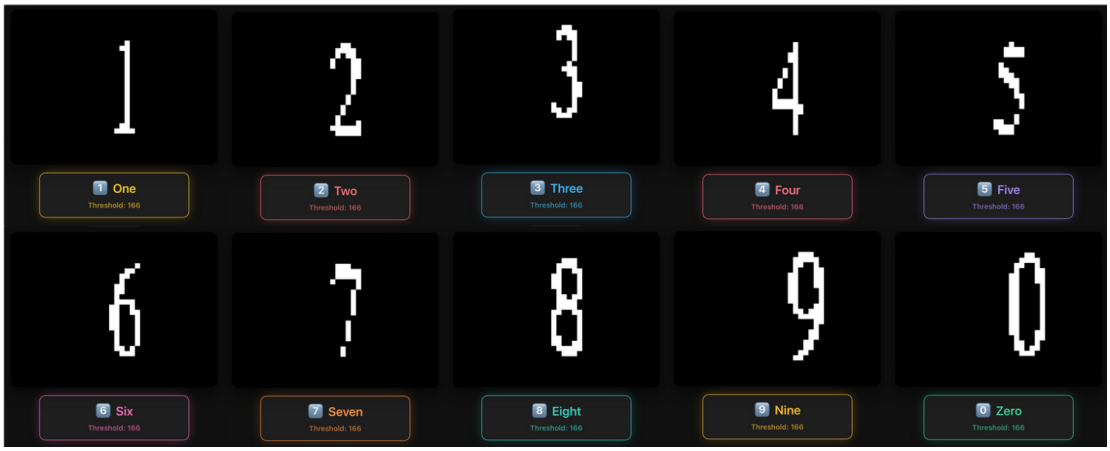

Figure 8.2: Functioning Camera-to-Prediction-to-Server Pipeline

All work described thus far has been documented in the repository linked on the title page. Methods were recorded and automated for reproducibility in a commercial environment. Development resulted in robust firmware, scalable verified RTL, a cohesive Pytorch-Hardware landscape featuring a complete EDA pipeline, and an exhaustive design space exploration. Future work should focus on remote deployment via battery power, learning a wider zero region for ternary activations to improve quantized accuracy, and exploring if predictive functions scale across computer vision tasks and FPGA architectures.

Bibliography

[1] J. Choi, Z. Wang, S. Venkataramani, P. I.-J. Chuang, V. Srinivasan, and K. Gopalakrishnan. Pact: Parameterized clipping activation for quantized neural networks, 2018. arXiv. https://doi.org/10.48550/arXiv.1805.06085.

[2] Espressif Systems. *ESP32-C3 Series Datasheet*, 2023. https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32c3/esp-dev-kits-en-master-esp32c3.pdf.

[3] A. Forencich. cocotbext-uart (version 0.1.4) [computer software], 2025. GitHub. https://github.com/alexforencich/cocotbext-uart.

[4] Lattice Semiconductor. *iCE40 LP/HX Family Data Sheet*. https://www.latticesemi.com/-/media/LatticeSemi/Documents/DataSheets/iCE/iCE40-LP-HX-Family-Data-Sheet.pdf.

[5] Lattice Semiconductor. *iCE40 Technology Library*, 2017. https://www.latticesemi.com/-/media/LatticeSemi/Documents/UserManuals/1D/SBTICETechnologyLibrary201701.pdf.

[6] J. H. Lee, S. Shin, V. Kim, J. You, and A. Chen. Unifying block-wise ptq and distillation-based qat for progressive quantization toward 2-bit instruction-tuned llms, 2025. arXiv. https://doi.org/10.48550/arXiv.2506.09104.

[7] UCTRONICS. *ArduCAM-M-2MP Camera Shield Datasheet*, 2017. https://www.uctronics.com/download/Amazon/ArduCAM_Mini_2MP_Camera_Shield_DS.pdf.

[8] R.-J. Zhu, Y. Zhang, S. Abreu, E. Sifferman, T. Sheaves, Y. Wang, D. Richmond, S. B. Shrestha, P. Zhou, and J. K. Eshraghian. Scalable matmul-free language modeling, 2024. arXiv. https://doi.org/10.48550/arXiv.2406.02528.